# Guest Identity, Reservation & Digital Key Validator Smart Contract

## Table Of Contents

## Overview

This unified Plutus V2 smart contract manages a guest's complete hotel stay life cycle, including identity submission, room reservation, digital key generation and validation, and final check-out. All actions are authorized via admin-signed transactions and validated using a nested GuestDatum structure to reduce memory usage and ensure precise, efficient validation logic.

### *Why This Unified Version?*

Previously, the smart contract logic was distributed across multiple redeemers and separate validation flows:

- SubmitIdentity, VerifyIdentity, and ConfirmIdentity were distinct steps for identity management.
- ReserveRoom and ConfirmReservation were handled separately.
- Digital key functionality was not fully integrated and required additional check-in state tracking.

This design resulted in a bulky GuestDatum with flat fields (14+), and higher on-chain memory usage due to redundant checks across separate redeemers and also both the smart contracts were  taking as a whole more than 15 minutes.

***Key Improvements in the Unified Version:***

- **Compact Nested Types**: Identity, reservation, and key information are now grouped in nested types (IdentityInfo, ReservationInfo, and KeyInfo), allowing better memory layout and simplified equality checks.
- **Reduced Redeemer Count**: Consolidated into 4 high-level actions:
    - FullIdentitySubmit (merged identity flow)
    - FullReservation (merged reservation creation and confirmation)
    - GenerateAndValidateKey (digital key generation and validation)
    - CheckOut (final exit and cleanup)
- **More Efficient Validator Logic**: Logic for unchanged fields is encapsulated in helper functions, reducing boilerplate and improving validator clarity.
- **Simplified Off-chain Logic**: With fewer redeemers and predictable field semantics, off-chain UTXO construction and updates are now easier to reason about  and implement.
- **Aligned with Real-World Flow**: The unified design more closely reflects a natural operational workflow — reserve → verify → issue key → checkout — with intermediate validation points built in.

This update makes the contract more scalable, testable, and maintainable across the full guest reservation and identity validation lifecycle.

## Data Structures

### Datum: GuestDatum

Represents the complete state of a guest's interaction with the system.

| Field | Type | Description |
|---|---|---|
| guestAddress | BuiltinByteString | Guest’s unique ID/address |
| adminPKH | PubKeyHash | Admin wallet that must sign transactions |
| identity | IdentityInfo | Identity fields nested in structure |
| reservation | ReservationInfo | Reservation details |
| keyInfo | KeyInfo | Digital Key , check-in-state , and flags |

**IdentityInfo**

| Field | Type | Description |
|---|---|---|
| name | BuiltinByteString | Guest name |
| passportNumber | BuiltinByteString | Passport number |
| photoHash | BuiltinByteString | Hash of identity photo |
| isUserVerified | Bool | Set true by admin |
| identityStatus | Bool | True if identity setup complete |

**ReservationInfo**

| Field | Type | Description |
|---|---|---|
| isReserved | Bool | Whether reservation is initiated |
| reservationStatus | Bool | Whether reservation is confirmed |
| reservationId | BuiltinByteString | Unique reservation identifier |
| checkInDate | BuiltinByteString | Guest's check-in-date |

| | | |
|---|---|---|
| checkOutDate | BuiltinByteString | Guest’s check-out-date |

**KeyInfo**

| Field | Type | Description |
|---|---|---|
| initiateCheckIn | Bool | True if guest is ready to check in |
| DigitalKey | BuiltinByteString | Digital key issued to the guest |
| isDigitalKeyValidated | Bool | Validation Status of the digital key |

## Redeemer: GuestRedeemer

Used to determine which part of the logic the validator should enforce.

- FullReservation
- FullIdentitySubmit

- GenerateAndValidateKey
- CheckOut

## Validation Logic by Redeemer

### 1.FullReservation

**Actor:** Admin only
**Purpose:** Sets room reservation data for the guest.

**Preconditions:**

- Guest address must be non-empty
- No existing reservation (isReserved == False)

**Expected Changes:**

- isReserved = True
- reservationStatus = True
- All reservation fields populated
- Other fields (identity, keyInfo, adminPKH) remain unchanged

### 2. FullIdentitySubmit

**Actor:** Admin only
**Purpose:** Captures guest identity data and marks them verified.

**Preconditions:**

- Guest must already have a reservation (isReserved == True)

- Existing identity fields must be empty

**Expected Changes:**

- Fields name, passportNumber, photoHash filled
- isUserVerified = True, identityStatus = True
- Other fields remain unchanged

### 3. GenerateAndValidateKey

**Actor:** Admin only
**Purpose:** Generates and validates the digital room key.

**Preconditions:**

- Guest has an active reservation (isReserved == True)
- Identity must be verified (isUserVerified == True)
- No digital key exists (digitalKey == "")

**Expected Changes:**

- digitalKey is populated
- initiateCheckIn = True
- isDigitalKeyValidated = True
- Other fields remain unchanged

### 4. CheckOut

**Actor:** Admin only
**Purpose:** Final checkout process to clear reservation and key info.

**Preconditions:**

- Guest must have been checked in (key and reservation were active)

**Expected Changes:**

- All reservation fields reset to default/empty
- digitalKey = "", isDigitalKeyValidated = False, initiateCheckIn = False
- Identity and admin fields remain unchanged

## Smart Contract Low-Level Diagram

https://drive.google.com/drive/folders/1-ZhYD2nWyAH_k7zIyTv42eteB8uhiu59

*please download and open it in the browser